Steady State Estimation

Efficient Parameter Estimation via Partial Convergence

Ed Hill (AA) and Jamie Lenney (MMD)

March 18, 2026

Introduction

- Despite significant advances a pain point for working with Heterogeneous Agent models can be the calibration of the Steady State
  - This has typically limited the amount of parameters estimated at the steady state and analysis around those parameters
- Today we propose an approach that can significantly speed up this process
  - Importantly for complex models like UK-HANK
  - But also simpler models without specialized soln methods e.g. VFI
- And have packaged our method(s) into python scripts to work in the SSJ framwork

Environment we are working in

- Two step solution method typical of DSGE literature
  1. Solve for steady state, targetting some moments [Focus today]
  2. Solve dynamics around the moments [Normal methods apply e.g. SSJ]
- E.g. Solving for β to hit a target interest rate r
  - Typical approach would be to fully solve a succession of steady states e.g. via a bisection method

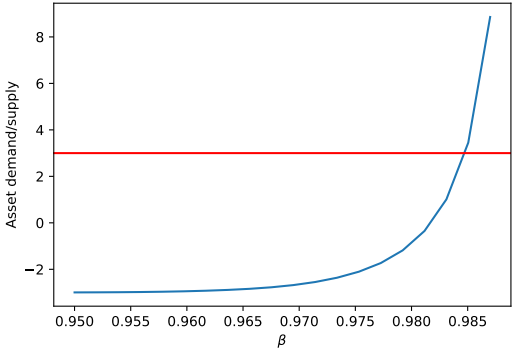


Figure Hugget model Saving supply / Demand

Environment: Krusell–Smith Model

Household: Solve Bellman equation for $V(a, e)$, policy $a'(a, e)$:

$$V(a, e, \chi) = \max_{c, a'} u(c) + \beta \mathbb{E}[V(a', e', \chi') \mid e], \quad c + a' = (1 + r)a + we$$

Firms: Competitive factor prices:

$$r = \alpha Z(K/L)^{\alpha - 1} - \delta, \quad w = (1 - \alpha)Z(K/L)^{\alpha}$$

Market Clearing: Saving equals next period capital stock

$$A \equiv \int a \, dD(a, e) \; = \; K'$$

Note: For solving Steady State, each evaluation of $A(\beta)$ requires:

- Backward iteration: converge $V(a, e)$ (typically 1000+ iterations)
- Forward iteration: converge distribution $D(a, e)$ (typically 1000+ iterations)

Challenges: Complex steady states

More complicated models start to create more interesting non-linear interactions.
Difficult to search over, especially with multiple targets.

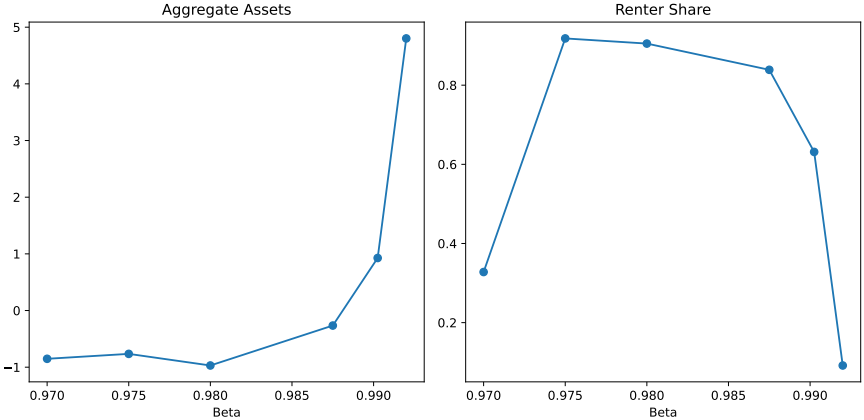


Figure UK-HANK

Challenges: Parameter danger zones

Issue for all models and not clear where these are in more complex models with less well understood parameters.

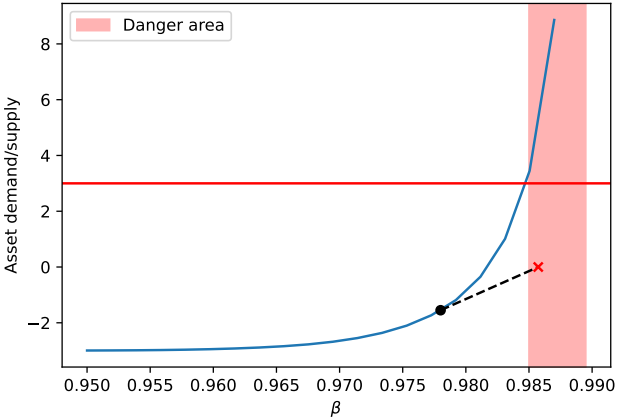


Figure Saving Schedule in Krusell Smith

Key idea

Don't fully solve the model while searching: Partially converge, forecast and adjust along the way

- Far fewer backward and forward steps
- Robust to danger zones
- Quickly learn the relevant and interesting areas of the parameter space
- Only fully converge value functions and market clearing at the end. [full convergence is costly]

Key idea

Standard method: Repeated value function convergence between discrete target improvements.

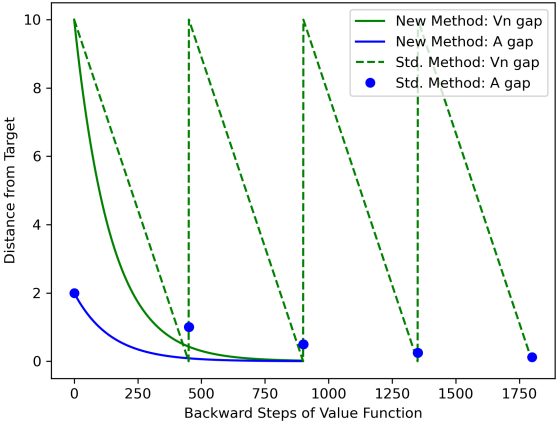


Figure Partial Convergence

Literature

Aggregate dynamics	Policy fn / SS speed	Global / non-linear
Reiter (2009) Linearisation around SS	**Carroll (2006)** Endogenous grid method	**Fernández-V. (2023)** Deep learning solution
Auclert et al. (2021) Sequence space Jacobian	**Winberry (2018)** Parametric SS calibration	**Maliar et al. (2021)** Deep learning DSGE
Ahn et al. (2018) Eigenvalue reduction	**This paper** Partial convergence + forecasting Works with any SS solver Additive to SSJ	**Elenev et al. (2021)** Calibration as policy fn
Bhandari et al. (2023) Non-linear perturbation		
Achdou et al. (2022) Continuous-time HA (HJB)		

Our approach is complementary to all three columns — it speeds up the steady-state step that each depends on.

Plan for rest of the talk

- **Two key ideas**: Partial convergence and forecasting
- **Two methods**: Gradient-based (Gauss-Newton-type) and Gradient-free
- **Demonstrate** key performance metrics using models ranging from simple ones to our UK-HANK policy model

Approach Overview: Partial convergence

- Given parameters X, define an **update** as
  - **Backwards update**: $V_n \to V_{n+1}$ given X using a 'backward step' solution of the Bellman equation or equivalent (e.g. Euler equation in $\partial_a V$)
  - **Forwards update**: $D_n \to D_{n+1}$ given X, V_{n+1} provide the actions and law-of-motion
  - **Moment calculation**: Use V_{n+1}, D_{n+1} to calculate Y_{n+1} - the set of moments and other derived quantities we are targetting.

- **Full convergence**: If X is held constant, $V_n, D_n \to V_\infty, D_\infty$
- **Partial convergence**: But we can terminate after n updates.

- **Variable** X: Usually X is held constant. But X can vary, X_n.

Approach Overview: Forecasting

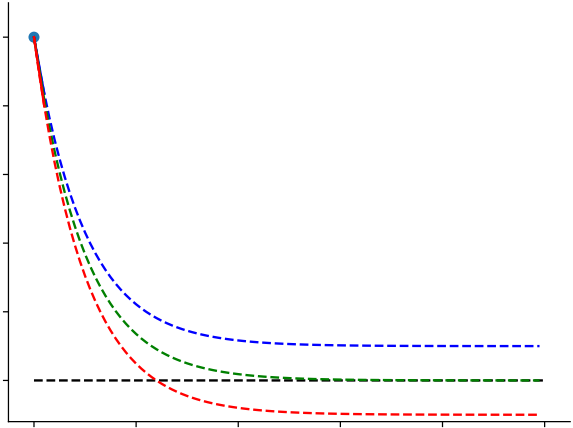


Figure Like bringing a plane into land: We can alter from red or blue to follow the green line.

Approach Overview: Forecasting

We expect exponential decay of $Y_n(X) \to Y_\infty(X)$, specifically we let:

$$Y_n(X) = y_n(X) + a$$

$$Y_{n+1}(X) = y_n(X) + ae^{-b}$$

$$Y_{n+2}(X) = y_n(X) + ae^{-2b}$$

$y_n(X)$ is the forecast of $Y_\infty(X)$.

Forecasting

- Type I: Estimate b (over time, from D, V). Use Y_n, Y_{n+1} to find $a, y_n(X)$.
  - $a = (Y_n - Y_{n+1})/(1 - e^{-b}), y_n = Y_n - a$
- Type II: Estimate $a, b, y_n(X)$ jointly from $Y_n, Y_{n-1}, \ldots Y_{n-k}$ $(k \geq 2)$

The error in this estimate, $|y_n(X) - Y_\infty(X)|$, is substantially lower than $|Y_n(X) - Y_\infty(X)|$ leading to increased speed and stability in the methods below.

Method I: Gauss-Newton Algorithm with forecasting

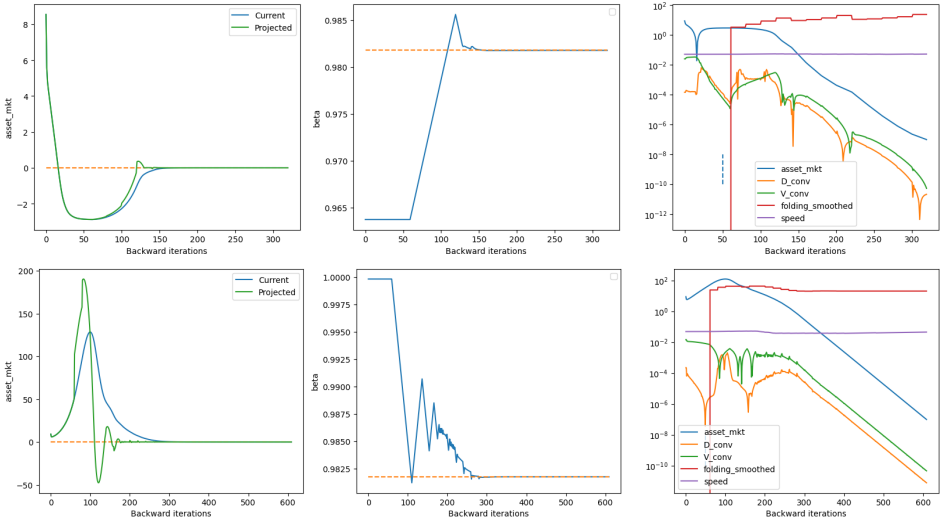


Figure Method I: The target (left), parameter (centre), and convergence (right) in the KS model

Method I: Gauss-Newton Algorithm with forecasting

Initialise, burn in starting point X_0, with V_0, D_0 compute initial moments Y_0

1. Perform V-update: $V_n, X_n \rightarrow V_{n+1}$ and D-update $V_{n+1}, X_n \rightarrow D_{n+1}$
2. Recalculate moments, project to estimate y_n
3. if $n \% n_d = 0$ **Compute Jacobian:** For i enumerating k parameters
   - Perform V-update: $V_n, X_n + \delta e_i \rightarrow V'_{n+1}$
   - Perform D-update $V'_{n+1}, X_n + \delta e_i \rightarrow D'_{n+1}$
   - Recalculate moments, project (Type I) to estimate y'_n
   - Find $J_i = (y'_n - y_n)/\delta$

   construct $J = \{J_0, \ldots, J_{k-1}\}$
4. **Gauss-Newton step:**

$$X_{n+1} = X_n - \alpha(J^T J)^{-1} J^T y_n \qquad (or\ \ X_{n+1} = X_n - \alpha J^{-1} y_n\ \ )$$

Repeat until $\|Y_{n+1} - Y_n\| < \epsilon_s$

Method II

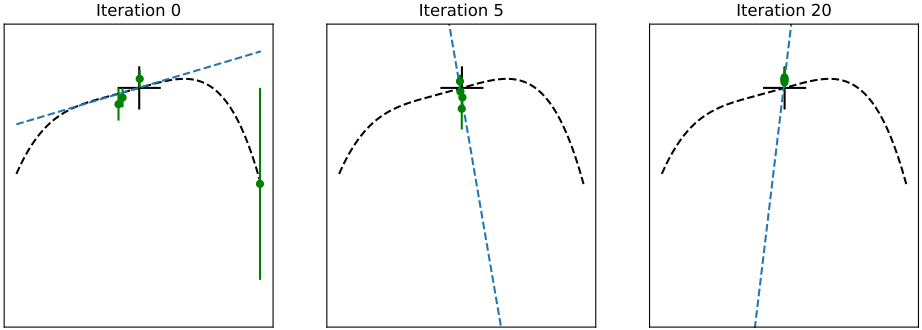


Figure Method II finding a solution on a non-linear and bumpy optimisation surface

Method II

Initialise, burn in population size N with X_i, $V_0^{(i)}$, $D_0^{(i)}$

1. **For each** X_i: compute $froc_i = \left| \frac{y_{n+1}(X_i) - y_n(X_i)}{y_n(X_i)} \right|^2$. If $froc_i > \epsilon_c$, X_i is **active**:
   - Perform V-updates and D-updates $\rightarrow V_{n+1}^{(i)} \rightarrow D_{n+1}^{(i)}$
   - Recalculate moments, project (Type II) to estimate $y_n(X_i)$, recompute $froc_i$
2. **Compute weights:** $w_i^{-1} = (\epsilon_f + froc_i) \cdot y_n(X_i)^2$
3. **WLS:** for each target j, fit $m_j X + c_j$ by minimising $\sum_i w_i \left(y_n^{(j)}(X_i) - (m_j X_i + c_j)\right)^2$
4. **Find candidate:** $X' = \arg\min_X \sum_j (m_j X + c_j)^2$
5. **Damped update:** let $X_{\text{best}} = \arg\max_i w_i$, then $X_{\text{new}} = \alpha X' + (1-\alpha) X_{\text{best}}$, $\alpha \sim \mathcal{U}(0.5, 1)$
6. **Update population:** append X_{new}, or replace $X_{\text{worst}} = \arg\min_i w_i$ to keep N constant

Key results

Look at performance over a few key determiners of the complexity of the problem

- Proximity to the solution of the initial guess
- Complexity of the model
  - Number of parameters being estimated (n_p)
  - Bumpy optimisation surface
  - Computational time of the backwards update

Models:

- A 1-asset Krusell-Smith model ($n_p = 1$)
- A 2-asset (liquid & illiquid) HANK model ($n_p = 2$)
- A 1-asset model with labour choice ($n_p = 2$) - with and without taste shocks
- A policy model: UK-HANK - 1-asset + a rent/flat/house housing choice ($n_p \leq 5$)

Metrics

- Successful completion
- Number of V evaluations
  - Harmonic mean
  - versus numbers to find steady state with known optimal parameters
- Wall time

Results: Accuracy of initial guess - KS model

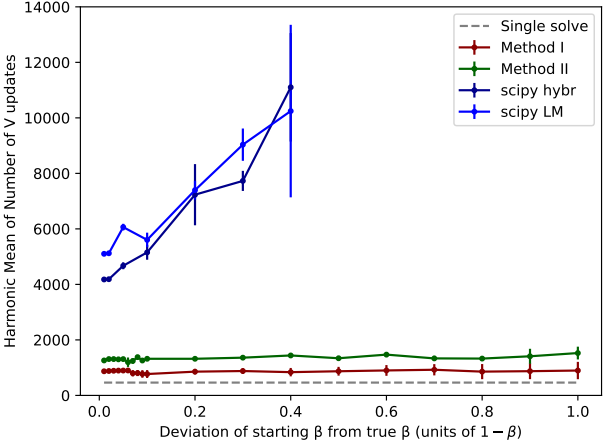


Figure Number of backwards iterations to calibrate a KS model. One parameter (β) is varied to hit one target (*asset_market*).

Results: Number of parameters - UK-HANK

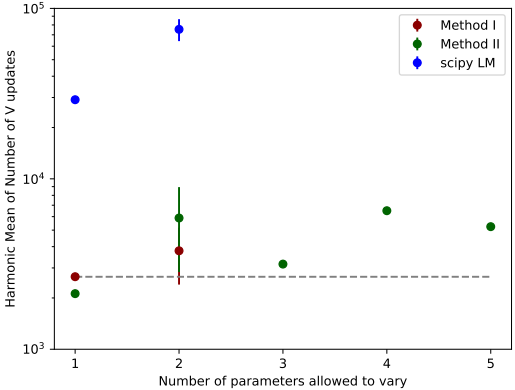
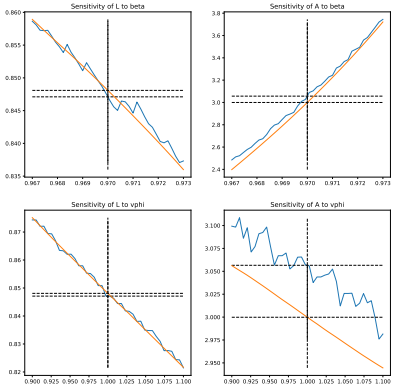


Figure Backwards iterations to solve the UK-HANK model with different subsets of parameters.

Notes $n_p \geq 3$ models are solved to a tolerance of 10^{-3}. For $n_p = 2$, both Method I and scipy LM have fails.

Results: Smoothness of optimisation surface - Labour choice



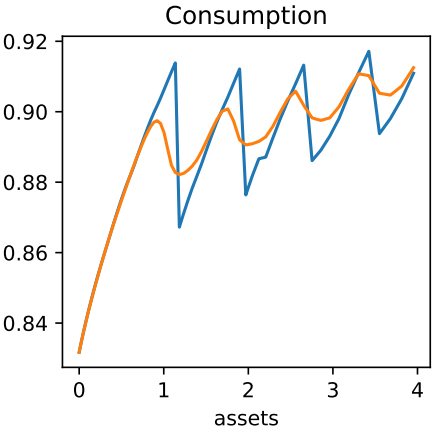


Figure (Left) Visualising the optimisation surface and (Right) the agents' consumption. Without (blue) and with (orange) taste shocks.

Results: Smoothness of optimisation surface - Labour choice

	Single solve	Method I	Method II		scipy LM
Tolerance			10^{-3}	10^{-10}	
Labour choice (with taste shock)	562	813	871	1752	10664
Labour choice (no taste shock)	562	-	1396	x	-
2-asset HANK	852	1142	-	3169	15341

Table Number of backwards iterations for different models with different methods.

Notes Labour choice (no taste shock): `scipy LM` fails once and terminates 3 times, after an average of 50,000 backward steps, but never with a sensible solution. `Method I` also fails. The surface creates a number of local minima very near the solution.

Results: Wall time - Backwards update complexity

Model	Time per V update	Solution time Method I	Method II	scipy LM	Single solve
KS	6.6×10^{-5}	3.7	5.5	**0.37**	0.03
Labour choice w/ t.s.	6.5×10^{-4}	**5.9**	11	7.0	0.66
2-asset HANK	1.6×10^{-3}	**9.5**	45	25	1.5
UK HANK ($n_p = 1$)	4.7×10^{-2}	352	**325**	1345	100
UK HANK ($n_p = 2$)	4.7×10^{-2}	520	712	4000*	100

Table Times are wall time in seconds on a laptop CPU (4c/8th i5). x (y) denotes $x \times 10^{-y}$. The closure of the model typically takes 5×10^{-3}s

Notes * Fails 50% of tests. The $n_p = 2$ case has a long, shallow minimum.

Questions

- **The lines look very flat**:
  - Yes. Because a lot of effort is spent in the convergence from $\sim 10^{-2}$ to 10^{-10}, burn-in of the solutions adds a distance/dimension independent constant, and we intentionally use the same hyperparameters across all runs.

- **Is convergence guaranteed**:
  - For Method I, yes, with a single global minimum. For Method II, no, but empirically it performs well.

- **Could you do better**:
  - Not much. The number of steps is often within a factor-of-two of converging V and D taking X as given, so there isn't much room for improvement. If working on one model repeatedly you could choose hyperparameters, or you could combine the two methods.

- **Is the code available**:
  - Yes. Happy to share and discuss internally, and it will be released with the paper.

Conclusion

About the method...

- Partial convergence with forecasting significantly reduces computational cost in both gradient-based and gradient-free optimisers
- Method II in particular is robust to complex and bumpy optimisation surfaces
- Performance scales favourably with model complexity
- The methods are practical and accessible - additive to the SSJ framework

... and its implications

- We can calibrate UK-HANK's steady-state
- Requires $\sim 10\times$ fewer evaluations of the backward iteration
  $\rightarrow$ More complex models, less optimised/more costly backward functions
- More robust
  $\rightarrow$ More confidence using complex models in policy